Lab 7 — Advanced SQL Synthesis & Analytics

Module I & II · Medium-Advanced SQL Practice · Duration: 2 hours
Tools: PostgreSQL / SQLite · **Deliverables:** `queries.sql`, `lab07_report.md`

Objectives

This lab provides medium-to-advanced practice on core SQL querying, multi-table aggregations, window ranking & framing functions, recursive network traversal, correlated subqueries, and database views using the **FinPay** digital wallet and risk engine platform:

1. **Multi-table Cash Flow Aggregations & Conditional Logic** (`GROUP BY`, `HAVING`, `CASE`).
2. **Window Ranking & Partition Market Share** (`DENSE_RANK()`, `SUM() OVER (PARTITION BY ...)`).
3. **Recursive Tree Traversal & Path Building** (`WITH RECURSIVE`).
4. **Interval-Based Velocity Spike Detection** (`LAG()`, Date/Time subtraction, `JULIANDAY`).
5. **Security & Audit Views** (`CREATE VIEW`, `LEFT JOIN`, `COALESCE`, `NOT EXISTS`).

Database Schema (FinPay Platform)

FinPay operates a digital payments, P2P transfers, and merchant acquiring platform. The database schema consists of 5 core tables:

```
users(user_id PK, name, email, kyc_status ['verified','pending','flagged'], signup_date, referrer_id FK
accounts(account_id PK, user_id FK, account_type ['savings','checking','wallet'], balance, created_date
merchants(merchant_id PK, merchant_name, category ['Retail','Travel','Food','Utilities','Entertainment'
transactions(txn_id PK, sender_account_id FK, receiver_account_id FK, merchant_id FK, txn_date, txn_tim
risk_alerts(alert_id PK, txn_id FK, risk_score [1-100], flag_reason, alert_date)
```

Setup (5 min)

1. Load the schema and sample dataset into your database environment (SQLite or PostgreSQL):

```
sqlite3 lab7.db < data/schema.sql
sqlite3 lab7.db < data/data.sql
```

2. Verify that all 5 tables are populated:

```
SELECT table_name FROM information_schema.tables WHERE table_schema='public'; -- PostgreSQL
-- OR --
.tables -- SQLite
```

Part 1 — Task 1: Monthly Account Cash Flow & Liquidity Ratio (Multi-Table Aggregations)

Business Context: Financial auditors need a monthly cash flow summary for user accounts, categorizing accounts by liquidity performance.

Task Requirements: 1. Join `accounts`, `users`, and `transactions` (both where account is sender or receiver). 2. Compute `total_inflow`: Sum of incoming transaction amounts (`txn_type IN ('deposit')` OR `receiver_account_id = account_id` for P2P) across successful transactions (`status = 'success'`).

3. Compute `total_outflow`: Sum of outgoing transaction amounts (`txn_type IN ('withdrawal', 'merchant_payment')` OR `sender_account_id = account_id` for P2P) across successful transactions. 4. Compute `net_cash_flow`: `total_inflow - total_outflow`. 5. Assign `liquidity_status`: - `'Surplus'` if `net_cash_flow > 0` - `'Deficit'` if `net_cash_flow < 0` - `'Balanced'` if `net_cash_flow = 0` 6. Filter out accounts with 0 activity (`total_inflow > 0 OR total_outflow > 0`). 7. Order results by `net_cash_flow` DESC.

Part 2 — Task 2: Merchant Category Leaders & Market Share (Window Ranking & Partition Aggregation)

Business Context: Merchant relationship managers want to identify top-performing merchants in each category and calculate their market share percentage within their category.

Task Requirements: 1. Aggregate total transaction volume (`SUM(amount)`) per merchant across successful payments (`status = 'success'`). 2. Compute `category_total_volume`: Total volume across all merchants in that category using `SUM(merchant_volume) OVER (PARTITION BY category)`. 3. Compute `market_share_pct`: `ROUND((merchant_volume * 100.0 / category_total_volume), 2)`. 4. Rank merchants per category using `DENSE_RANK() OVER (PARTITION BY category ORDER BY merchant_volume DESC)`. 5. Filter to show only the **top 2 merchants per category** (`WHERE category_rank <= 2`). 6. Order results by `category` ASC and `category_rank` ASC.

Part 3 — Task 3: Referral Network Depth & Viral Reach (Recursive CTEs)

Business Context: Growth analysts want to trace multi-level user referral chains to identify top viral growth loops starting from seed users (`referrer_id IS NULL`).

Task Requirements: 1. Write a **Recursive CTE** (`WITH RECURSIVE`) starting from seed users without a referrer (`referrer_id IS NULL`). 2. Compute `referral_depth` (0 for seed users, 1 for direct invitees, 2 for second-level, etc.). 3. Build `referral_chain`: Path string of user names showing the full invite lineage (e.g. `'Aarav Sharma -> Liam Chen -> Sophia Martinez'`). 4. Filter for invitees at `referral_depth >= 1`. 5. Order results by `referral_depth` ASC and `name` ASC.

Part 4 — Task 4: Suspicious Velocity & Rapid Multi-Transaction Spikes (Window Functions & Time Intervals)

Business Context: Fraud prevention algorithms flag rapid consecutive transactions between accounts happening within short time windows.

Task Requirements: 1. Identify consecutive P2P transactions sent by the same account (`sender_account_id`). 2. Retrieve `prev_txn_time`, `prev_txn_date`, and `prev_amount` using `LAG() OVER (PARTITION BY sender_account_id ORDER BY txn_date, txn_time)`. 3. Compute `time_gap_seconds`: Difference in seconds between current and previous transaction (in SQLite: `CAST((JULIANDAY(txn_date || ' ' || txn_time) - JULIANDAY(prev_txn_date || ' ' || prev_txn_time)) * 86400 AS INTEGER)`). 4. Flag `velocity_risk_flag`: - `'RAPID HIGH-VALUE VELOCITY'` if `time_gap_seconds <= 300` (5 minutes) AND `(amount + prev_amount) >= 2000.00`. - `'NORMAL'` otherwise. 5. Filter out initial transactions (where `prev_txn_time IS NULL`). 6. Order results by `sender_account_id` ASC, `txn_date` ASC, and `txn_time` ASC.

Part 5 — Task 5: High-Risk Account Audit View (Database Views & Correlated Subqueries)

Business Context: Compliance officers need a real-time database View `v_flagged_account_audit` tracking accounts requiring immediate audit.

Task Requirements: 1. Create a database View named **`v_flagged_account_audit`**. 2. Select accounts belonging to users who are **NOT fully KYC verified** (`kyc_status != 'verified'`). 3. Include accounts that satisfy **EITHER** of the following conditions: - Associated with **at least 1 failed or reversed transaction** (`status IN ('failed', 'reversed')`). - Associated with **at least 1 risk alert** with a `risk_score >= 80`. 4. Display `account_id`, `user_name`, `kyc_status`, `account_type`, `balance`, `failed_or_reversed_txn_count`, and `max_risk_score`. 5. Order output by `balance` DESC. 6. Execute `SELECT * FROM v_flagged_account_audit;` to verify your view.

Submission Checklist

Your submission folder `lab07/` must contain: - **`queries.sql`** — Containing your tested SQL queries for Tasks 1–5, clearly commented. - **`lab07_report.md`** — Short report containing your output tables for Tasks 1–5 and a 2-sentence explanation of the SQL mechanics for each.